DSA revision companion

Structures you can draw. Algorithms you can explain.

39 lessons / 13 modules / 130 topic questions / 65 flashcards

Use this concise companion for retrieval after studying the full lessons. The downloadable HTML notes contain all explanations, worked code, exercises and solutions. The interactive site adds editable Python and 19 step-through visual models.

Official scope verified

Every named Programming & DSA topic in Section 4 of the official GATE DA 2027 syllabus is mapped in the site. Python is taught in the existing track. DSA covers stacks, queues, linked lists, trees, hash tables, both named searches, all five named sorts, graph fundamentals, traversals and shortest paths.

Complexity, invariants and implementation details support those topics. The final module on MSTs, union-find, dynamic programming and backtracking is enrichment. This is not a guide to the other DA syllabus sections.

Verified 24 October 2026. Original lessons and conceptual questions; official PYQs are identified by year/question number and linked to their full papers and answer keys.

Read the official GATE DA 2027 syllabus

Study order

01. Algorithms, correctness & cost

Define a contract, prove an invariant, count time and space

02. Arrays, stacks, queues & deques

Choose representations and implement ADT operations safely

03. Linked lists & pointer reasoning

Build, insert, delete, reverse and detect cycles

04. Hash tables & collision strategies

Trace chains, probing, deletion and load factor

05. Searching & boundary reasoning

Linear search, binary search, duplicates and monotone predicates

06. Selection, bubble & insertion sort

Trace passes, count inversions and distinguish stability

07. Merge sort & quicksort

Divide-and-conquer, partition invariants and complexity tradeoffs

08. Trees, traversals & search trees

Understand structure, reconstruct traversals and maintain a BST

09. Heaps & priority queues

Tree-backed priorities, heapify and heapsort

10. Graph fundamentals & representations

Vertices, edges, connectivity and storage choices

11. BFS, DFS & topological order

Trace frontiers, timestamps, cycles and dependency ordering

12. Shortest-path algorithms

Choose BFS, Dijkstra, DAG relaxation, Bellman-Ford or Floyd-Warshall

13. Advanced patterns & practical projects

Greedy reasoning, dynamic programming and a route planner

Complexity and assumptions

Costs below use bounded-size arithmetic and adjacency lists unless stated. h is tree height; V and E count vertices and edges; alpha is hash-table load factor. Input storage is excluded from auxiliary space.

Operation / algorithm	Time	Extra space / condition
Array indexed access	O(1)	Random access
List append	O(1) amortised	Single resize may cost O(n)
List front insertion/removal	O(n)	Suffix shifts
Stack push/pop; deque ends	O(1) amortised / O(1)	Array stack / deque
Linked insertion after known node	O(1)	Locating it may cost O(n)
Hash-chain lookup	Expected O(1+alpha)	Worst O(n) with collisions
Linear search	Best O(1), worst O(n)	O(1)
Binary search	Worst O(log n)	O(1) iterative; sorted array
Selection sort	Theta(n^2) all cases	O(1); unstable
Bubble with early stop	Best Theta(n), worst Theta(n^2)	O(1); stable with strict swap
Insertion sort	O(n+I), worst Theta(n^2)	O(1); I inversions; stable
Merge sort	Theta(n log n)	O(n) typical; stable merge
Quicksort	Expected O(n log n), worst O(n^2)	Stack O(log n) expected; unstable
BST search/insert/delete	O(h)	h may be n-1
Heap peek / insert / extract	O(1) / O(log n) / O(log n)	Complete binary tree
Bottom-up heapify	O(n)	O(1) with iterative sift
Heapsort	O(n log n)	O(1); unstable
BFS / DFS / topological	O(V+E)	O(V)
Dijkstra with decrease-key heap	O((V+E) log V)	O(V); nonnegative weights
Dijkstra lazy heap (lesson code)	O((V+E) log(E+1))	Up to O(E+V) storage
DAG shortest paths	O(V+E)	O(V); negative edges allowed
Bellman-Ford	O(VE)	O(V); detects reachable negative cycles
Floyd-Warshall	O(V^3)	O(V^2); all pairs
0-1 BFS	O(V+E)	O(V); weights only 0 or 1

Algorithms, correctness & cost

Define a contract, prove an invariant, count time and space

01 / From a problem to a correct algorithm

- An algorithm is a finite, unambiguous method for solving a specified problem.

- Correctness has two parts: partial correctness says the result is right if the procedure terminates; termination says it eventually stops.

- For a running maximum, after processing the first k elements, best is the maximum of exactly those k elements.

- Testing samples cannot prove correctness for all inputs, but carefully selected tests expose errors.

Trap: A plausible-looking output is not a proof. State exactly which prefix the invariant describes.

Recall task: Implement minimum with the same contract without slicing.

02 / Asymptotic analysis and loop counting

- Measure cost as a function of input size n under a stated model: count comparisons, assignments or other unit-cost operations.

- For a fixed input length, best case is the minimum cost over valid inputs and worst case the maximum.

- Sequential loops add costs; truly nested full loops multiply.

- Drop constants and lower-order terms for asymptotic growth, but retain exact counts when a NAT question asks for them. log bases differ by constants.

Trap: An upper bound need not be tight: a linear function is also O(n^2). Prefer Theta when the question asks for a tight bound.

Recall task: Count all ordered pairs i,j with 0 <= i,j < n for n=5.

03 / Recurrences, stack space and amortisation

- Recursive cost includes the work outside recursive calls and all recursive children.

- For T(n)=aT(n/b)+Theta(n^d), compare a with b^d: smaller means Theta(n^d), equal means Theta(n^d log n), larger means Theta(n^(log_b a)), under the usual balanced-subproblem assumptions.

- Time and live memory are different.

- Amortised cost bounds a whole operation sequence without a probability assumption.

Trap: Space from recursive slices can exceed the call-stack-only bound. Analyse the implementation actually shown.

Recall task: Write a recurrence for two half-sized calls doing n merge operations and compute total merge work for n=8.

Arrays, stacks, queues & deques

Choose representations and implement ADT operations safely

01 / Arrays, dynamic lists and operation costs

- An abstract data type specifies behaviour; a data structure specifies a representation.

- A list append is amortised O(1); pop() at the end is O(1).

- Dynamic capacity is not the same as logical length.

- A prefix-sum array P with P[0]=0 and P[i+1]=P[i]+a[i] answers the sum of a[l:r] as P[r]-P[l] in O(1), after O(n) preparation.

Trap: Python list.pop(0) is linear, even if your queue algorithm uses only one such operation per iteration.

Recall task: Answer inclusive range [1,2] using prefix sums.

02 / Stacks: LIFO and expression processing

- A stack is last-in, first-out. push adds at the top, pop removes the top, and peek reads it.

- Balanced brackets are a stack problem because the most recently opened bracket must be the first closed.

- For postfix evaluation, push numbers and, on an operator, pop the right operand first and then the left.

- For stack permutations of input 1,2,...,n, simulate pushes until the desired next output is on top, then pop.

Trap: For subtraction and division, reversed pop order changes the answer.

Recall task: Evaluate postfix tokens 8 3 - 2 * with a stack.

03 / Queues, circular buffers and deques

- A queue is first-in, first-out: enqueue at the rear and dequeue from the front.

- A circular array queue reuses slots with modulo arithmetic.

- A queue implemented by two stacks pushes arrivals into an input stack.

- Deques also support monotonic-window algorithms: remove expired indices at the front and dominated values at the back.

Trap: Equal head and tail indices alone are ambiguous unless your full/empty convention resolves them.

Recall task: Use deque to enqueue 4,5, remove 4, add 6 and list the queue.

Linked lists & pointer reasoning

Build, insert, delete, reverse and detect cycles

01 / Nodes, head pointers and insertion

- A singly linked list is a chain of nodes containing data and a next reference. head refers to the first node; None ends an acyclic list.

- Prepending a node is O(1): set new.next to the old head, then set head to new.

- Appending is O(1) with a maintained tail pointer, otherwise O(n) to locate the end.

- A doubly linked list adds prev references, making deletion of a known node O(1) if head/tail updates are handled.

Trap: Overwriting p.next before saving the old successor can disconnect the remaining suffix.

Recall task: Build the chain 5 -> 7 -> 9 by prepending values in reverse order.

02 / Deletion and iterative reversal

- To delete a node by value in a singly linked list, keep a predecessor while searching.

- Deleting a known non-tail node by copying its successor's data is a specialised trick, not a general deletion operation: it changes node identity semantics and cannot delete a tail this way.

- Iterative reversal maintains prev, current and next.

- Recursive reversal can be elegant but uses O(n) call frames, and long lists can exceed Python's recursion limit.

Trap: Updating current before preserving its next link can lose either the suffix or the node being reversed.

Recall task: Remove the first node with value 2 from 1 -> 2 -> 3 using a sentinel.

03 / Fast/slow pointers and cycle detection

- Two pointers at different speeds solve linked-list questions without indexing.

- Floyd's cycle algorithm starts slow and fast at the head and repeatedly advances them by one and two.

- To locate the cycle entry after a meeting, reset one pointer to head and advance both one step until they meet again.

- To measure cycle length, keep one pointer at the meeting node and walk another until it returns.

Trap: The entry-finding phase is valid only after confirming a meeting. An acyclic list needs a separate exit path.

Recall task: Find the second middle of [1,2,3,4] using slow/fast pointers.

Hash tables & collision strategies

Trace chains, probing, deletion and load factor

01 / Hash functions, load factor and chaining

- A hash table maps keys to bucket indices with a hash function.

- Separate chaining stores all keys mapping to a bucket in a list or another collection.

- Insertion may first search for an existing key if duplicate keys are disallowed or values must be updated.

- Python dict and set provide useful practical hash-table interfaces, but exam problems often specify a custom h, capacity and collision scheme.

Trap: Expected O(1) requires assumptions about hashing and controlled load; it is not a universal worst-case guarantee.

Recall task: Delete 7 from its chain and confirm 17 remains.

02 / Linear, quadratic and double probing

- Open addressing stores entries in the table itself.

- Linear probing tends to create primary clusters: contiguous occupied runs attract further collisions.

- For double hashing, the step must be nonzero and relatively prime to m to visit all m slots.

- An open-addressed table needs unused capacity; as alpha approaches one, searches become costly.

Trap: A free slot somewhere is insufficient if your probe sequence cannot reach it.

Recall task: List a double-hash probe sequence for m=7, h1=3, h2=2.

03 / Deletion tombstones and practical frequency maps

- Search in open addressing follows the same probe sequence as insertion.

- Insertion can remember the first tombstone for later reuse, but a map that must update existing keys should continue probing to check that the key does not already exist farther along the sequence.

- Frequency maps are a direct application: for each value x, set count[x]=count.get(x,0)+1.

- For two-sum, look for target-x among earlier elements before inserting x.

Trap: Replacing a deleted probe slot with an ordinary empty marker can make a later present key appear absent.

Recall task: Count frequencies of a,b,a,c,b,a.

Searching & boundary reasoning

Linear search, binary search, duplicates and monotone predicates

01 / Linear search and sentinels

- Linear search inspects elements one at a time until it finds the target or exhausts the sequence.

- Decide whether the result is a Boolean, the first matching index, any matching index or all matches.

- A sentinel search places a guaranteed matching value at a boundary so the loop does not repeatedly test for the end.

- When the data is sorted, a linear search may stop after encountering a larger value.

Trap: If a question counts equality tests, count equality tests only; do not silently mix them with loop-condition tests.

Recall task: Return every matching index of 2 in [2,4,2,2].

02 / Binary search with a precise invariant

- Binary search requires a sorted random-access sequence and a comparison order consistent with that sorting.

- The invariant is that any existing occurrence of the target remains within the candidate interval.

- For the common mid=(lo+hi)//2 procedure that probes one element each iteration, the maximum number of probes on n>0 elements is floor(log2 n)+1.

- mid=lo+(hi-lo)//2 avoids overflow in fixed-width languages.

Trap: Binary search on unsorted input has no correctness guarantee even if one sample happens to succeed.

Recall task: Record the middle indices when searching for 12 in [1,3,5,8,12].

03 / Lower bounds, duplicates and binary search on answers

- A lower bound returns the first index whose value is at least x; an upper bound returns the first index whose value is greater than x.

- Use [lo,hi) with lo=0,hi=n.

- More generally, binary search finds the first true position of a monotone predicate with false values followed by true values.

- For minimum integer k with k*k>=x and x>=0, the predicate is monotone.

Trap: Any-match binary search does not automatically return the first duplicate.

Recall task: Implement upper bound and count occurrences of 3 in [1,3,3,3,7].

Selection, bubble & insertion sort

Trace passes, count inversions and distinguish stability

01 / Selection sort and sorting vocabulary

- Sorting arranges items by a key.

- Selection sort repeatedly finds the minimum of the unsorted suffix and swaps it into the first unsorted position.

- Standard swap-based selection sort is in-place and Theta(n^2) in best and worst cases, and generally unstable.

- When tracing an exam question, record the array after the requested pass, not after every comparison unless asked. "Pass" must refer to the outer loop.

Trap: A sorted-looking array does not cause standard selection sort to stop early.

Recall task: Count selection-sort key comparisons for n=6.

02 / Bubble sort and early termination

- Bubble sort compares adjacent elements and swaps an inverted pair.

- The optimised algorithm resets swapped=False at the start of each pass and sets it when a swap occurs.

- Swap only when left>right to preserve stability; swapping equal keys can destroy it.

- An inversion is a pair of indices i<j with a[i]>a[j].

Trap: Best-case Theta(n) applies only to a version with an early-stop check.

Recall task: List strict inversion pairs for [3,1,2,1].

03 / Insertion sort and nearly sorted data

- Insertion sort grows a sorted prefix.

- Using a[j]>key rather than >= retains the order of equal keys, so the algorithm is stable.

- If the input has I inversions, ordinary insertion sort runs in O(n+I), making it useful for small or nearly sorted runs.

- Selection, bubble and insertion all have quadratic worst cases, but their pass invariants differ.

Trap: Binary search for the insertion location does not eliminate the cost of shifting an array suffix.

Recall task: Count insertion shifts for reverse input [4,3,2,1].

Merge sort & quicksort

Divide-and-conquer, partition invariants and complexity tradeoffs

01 / Merge two sorted sequences and build merge sort

- Merging combines two sorted sequences using a pointer to each next unused item.
- Merge sort divides the input into two nearly equal parts, recursively sorts them and merges their results.
- To make merging stable, take the left item first on equal keys.
- Bottom-up merge sort avoids recursion by merging runs of width1,2,4,... until the width covers the input.

Trap: Taking from the right on equality can destroy stability even though numeric output remains sorted.

Recall task: Merge [1,4,8] and [2,4,9] iteratively.

02 / Quicksort and Lomuto partition

- Quicksort partitions around a pivot and recursively sorts the resulting sides.
- When a scanned value is <=pivot, swap it into index i and advance i.
- Balanced splits yield Theta(n log n); repeatedly splitting into 0 and n-1 gives Theta(n^2).
- Standard in-place quicksort is unstable.

Trap: Hoare and Lomuto partition return different boundaries. Never reuse one scheme's recursion bounds with the other.

Recall task: Complete recursive quicksort using this Lomuto partition.

03 / Duplicates, lower bounds and choosing a sort

- Two-way partition schemes can perform poorly with many equal keys.
- Any comparison sort on n distinct keys has n! possible orders to distinguish.
- Counting sort can run in O(n+k) for integer keys in a known range of size k, trading range-dependent space for fewer comparisons.
- Choose a sort using the needed guarantee: stable array sorting suggests merge sort or Python's stable built-in sort; tiny nearly sorted input favours insertion sort; predictable constant extra array storage may favour heapsort; quicksort is often fast but needs pivot care.

Trap: Do not describe this allocating demonstration as in-place just because its partition idea can be implemented in-place.

Recall task: Counting-sort nonnegative keys [3,1,3,0,2] with maximum 3.

Trees, traversals & search trees

Understand structure, reconstruct traversals and maintain a BST

01 / Tree terminology and traversal orders

- A tree is a connected undirected graph with no cycles.

- A binary tree allows at most two children, distinguished as left and right.

- For binary trees of height h, maximum nodes are 2^(h+1)-1, while a skewed tree of that height has h+1 nodes.

- Preorder visits root,left,right; inorder left,root,right; postorder left,right,root.

Trap: A complete tree need not be full, and a full tree need not be complete.

Recall task: Write a linear-time inorder traversal using an output list and append.

02 / Reconstructing trees and binary search insertion

- With distinct keys, preorder plus inorder uniquely determines a binary tree: the first preorder item is the root; locate it in inorder to separate left and right subtrees; recurse on matching segments.

- Preorder plus postorder does not uniquely determine an arbitrary binary tree, because a lone child may be left or right.

- A binary search tree (BST) has every left-subtree key less than the node key and every right-subtree key greater, assuming distinct keys.

- Search and insertion follow comparisons from the root and cost O(h).

Trap: Checking only child keys is insufficient: a descendant can violate an ancestor's bound.

Recall task: Reconstruct the root and left/right inorder segments for preorder A B D E C and inorder D B E A C.

03 / BST deletion, successor and balanced-tree ideas

- Deleting a BST node has three structural cases.

- The successor is the minimum of a nonempty right subtree.

- BST correctness depends on keeping all ancestor bounds valid.

- Balanced search trees maintain height logarithmic through structural rules.

Trap: Returning a changed subtree without assigning it back to its parent leaves the original tree unchanged.

Recall task: Delete a root with only one child and verify the child becomes the root.

Heaps & priority queues

Tree-backed priorities, heapify and heapsort

01 / Binary heaps and sift operations

- A min-heap is a complete binary tree where every parent key is no greater than its children.

- In a zero-based array, children of i are2i+1 and 2i+2 and its parent is(i-1)//2 for i>0.

- Peek is O(1), insert and extract are O(log n), and searching for an arbitrary value is O(n) in the worst case.

- Python heapq implements a min-heap.

Trap: A heap is not a BST: it has parent-child priority order but no left-smaller/right-larger search rule.

Recall task: Keep the largest three values of [4,8,1,7,6] in a min-heap.

02 / Bottom-up heap construction and heapsort

- Bottom-up heap construction sifts down every internal node from the last one toward the root.

- Although one sift-down can take O(log n), most nodes lie near the leaves.

- Heapsort builds a max-heap, swaps the root with the last active element, reduces the active heap size and restores order.

- The active heap boundary is separate from total array length.

Trap: Multiplying n by the maximum sift height gives a loose O(n log n) bound for heapify; summing actual heights yields the tight O(n) bound.

Recall task: Heapify [9,1,5,2] with heapq and verify each parent is no larger than its children.

Graph fundamentals & representations

Vertices, edges, connectivity and storage choices

01 / Graph vocabulary and counting facts

- A graph G=(V,E) consists of vertices and edges.

- An undirected simple graph on n vertices has at most n(n-1)/2 edges; a directed simple graph without self-loops has at most n(n-1).

- A walk may repeat vertices and edges; a path here means no repeated vertices; a cycle returns to its start without other repeated vertices.

- A directed acyclic graph is a DAG.

Trap: An undirected edge appears twice in an adjacency list but is still one edge in |E|.

Recall task: Compute maximum edges for a simple undirected graph with 6 vertices.

02 / Adjacency lists, matrices and edge lists

- An adjacency list stores each vertex's outgoing neighbours.

- An undirected adjacency matrix is symmetric.

- An edge list stores triples(u,v,w), useful when an algorithm scans all edges, as Bellman-Ford does.

- Representation changes traversal complexity: scanning every row of a matrix costs O(V^2), while adjacency-list BFS/DFS takes O(V+E).

Trap: A zero edge weight is a valid weight; do not confuse it with absence.

Recall task: Build directed adjacency lists for 0->1 and 2->1.

03 / Reachability, components and bipartite colouring

- To count connected components of an undirected graph, launch a traversal from every still-unvisited vertex and increment a counter each time.

- Bipartite testing assigns one of two colours.

- A tree is bipartite because root-depth parity gives a valid two-colouring.

- Directed reachability is not symmetric.

Trap: A graph with one bipartite component and a separate triangle is not bipartite.

Recall task: Count components of the undirected graph [[1],[0],[]].

BFS, DFS & topological order

Trace frontiers, timestamps, cycles and dependency ordering

01 / Breadth-first search and shortest-hop paths

- BFS explores in nondecreasing number of edges from a source.

- Store dist[source]=0 and dist[v]=dist[u]+1 when v is first discovered from u.

- With adjacency lists, each reachable vertex is enqueued once and each outgoing edge scanned once: O(V+E) time and O(V) extra space.

- BFS optimises the number of edges, which equals total cost only when all edges have the same positive weight.

Trap: Different valid BFS orders can yield different parent trees while retaining the same distances.

Recall task: Change neighbour order at 0 to[2,1] and predict parent[3].

02 / Depth-first search, finishing times and cycles

- DFS follows a branch as far as possible before backtracking.

- Use white for undiscovered, grey for active and black for finished.

- In an undirected simple graph, ignore the edge back to the immediate parent when detecting a cycle; that is the same tree edge viewed in reverse.

- DFS costs O(V+E) on adjacency lists and uses O(V) visited/stack space.

Trap: For directed cycle detection, "visited" alone loses the distinction between active and finished vertices.

Recall task: Trace recursive preorder DFS on 0->[1,2],1->[3],2->[],3->[].

03 / Topological sorting and dependency scheduling

- A topological order of a directed graph places u before v for every edge u->v.

- Kahn's algorithm counts indegrees and queues all zero-indegree vertices.

- DFS gives another method: in a DAG, reverse the finishing order.

- The order is unique if Kahn's algorithm has exactly one eligible choice at every step.

Trap: A partial Kahn output is not a topological order of the full graph when a cycle remains.

Recall task: Validate [1,0,2,3] against edges0->2,1->2,2->3.

Shortest-path algorithms

Choose BFS, Dijkstra, DAG relaxation, Bellman-Ford or Floyd-Warshall

01 / Relaxation, path reconstruction and Dijkstra

- A shortest path minimises total edge weight.

- Dijkstra repeatedly settles the reachable vertex with minimum tentative distance and relaxes its outgoing edges.

- Using a binary heap and adjacency lists, a decrease-key version takes O((V+E)log V).

- To reconstruct a reachable path, follow parents backward from target to source and reverse.

Trap: Choosing a neighbour with the smallest individual edge is not Dijkstra. Choose the globally smallest tentative total distance.

Recall task: Use relaxation to compare direct cost 10 with a path of edges3 and 4.

02 / Bellman-Ford and negative-cycle detection

- Negative edges can make a later route improve an earlier label.

- The dynamic-programming invariant is that after i full passes, distances are no worse than the best source paths using at most i edges.

- If an edge can still improve after V-1 passes, a negative cycle is reachable from the source.

- Bellman-Ford costs O(VE) time and O(V) distance/parent storage.

Trap: A negative edge is not itself a negative cycle in a directed graph.

Recall task: Detect the reachable negative cycle0->1 weight 1,1->2 weight-3,2->1 weight 1.

03 / Floyd-Warshall for all pairs

- Floyd-Warshall computes distances between every pair.

- At stage k, allow vertex k as an intermediate and update D[i][j]=min(D[i][j],D[i][k]+D[k][j]).

- Time is O(V^3), space O(V^2).

- To reconstruct paths, maintain a next-hop matrix and set next[i][j]=next[i][k] when an improvement via k occurs.

Trap: Arbitrarily permuting the three loops can break the allowed-intermediate invariant.

Recall task: Compute reachability closure for 0->1->2 with Boolean Warshall.

04 / DAG paths, 0-1 BFS and choosing the right method

- On a DAG, process vertices in topological order and relax all outgoing edges.

- When edge weights are only0 or1, 0-1 BFS uses a deque: push an improved neighbour to the front for weight 0 and to the back for weight 1.

- Choose by the input: equal positive weights -> BFS; DAG -> topological relaxation; arbitrary nonnegative weights -> Dijkstra; negative edges with source queries -> Bellman-Ford; dense all-pairs queries -> Floyd-Warshall.

- Shortest-path trees and minimum spanning trees solve different problems.

Trap: A negative edge does not rule out linear-time shortest paths when the graph is a DAG.

Recall task: Use 0-1 BFS to find distances in0->1(1),0->2(0),2->1(0).

Advanced patterns & practical projects

Greedy reasoning, dynamic programming and a route planner

01 / Greedy methods, disjoint sets and minimum spanning trees

- A greedy algorithm commits to a locally attractive choice.

- A minimum spanning tree (MST) connects a weighted undirected connected graph with minimum total edge weight.

- Union by size/rank plus path compression gives almost constant amortised disjoint-set operations, commonly written O(alpha(V)).

- The cut property justifies choosing a lightest edge across a cut respected by the current forest.

Trap: MST minimises total network cost, not each source-to-destination distance.

Recall task: Show the greedy coin counterexample for target6 using coins1,3,4.

02 / Dynamic programming and backtracking

- Dynamic programming stores answers to overlapping subproblems.

- For minimum coins to form amount s, dp[0]=0 and dp[s]=1+min(dp[s-c]) over usable denominations c<=s.

- Backtracking explores choices and reverses them when returning.

- Keep these techniques separate from the DA syllabus's named divide-and-conquer sorting: merge sort solves disjoint subproblems; coin change reuses overlapping states.

Trap: Caching by an incomplete state can return a correct answer for the wrong subproblem.

Recall task: Generate all subsets of [1,2,3] iteratively.

03 / Build a route planner and an exam-solving workflow

- Build a small campus or delivery-route planner.

- Use an adjacency list for storage, Dijkstra for fastest routes, and parent links for route reconstruction.

- For exam traces, write the representation and convention first: zero/one indexing, inclusive/half-open intervals, neighbour order, pivot scheme, and height definition.

- A useful mastery loop is predict -> trace by hand -> execute -> change an edge case -> explain the invariant without notes.

Trap: Checking only edges visited before an early exit is not a full input validation pass.

Recall task: Add input validation that rejects negative weights anywhere in the graph.

Choose the shortest-path method

- All edges have equal positive cost: BFS minimises hops.
- DAG: topological relaxation, even with negative edges.
- Nonnegative arbitrary weights: Dijkstra.
- Negative edges: Bellman-Ford for source paths; report reachable negative cycles.
- Dense all-pairs queries: Floyd-Warshall; a negative diagonal identifies a negative cycle.
- Weights only 0 or 1: deque-based 0-1 BFS.
- An MST minimises total network weight, not source-to-target distances.

Exam trace checklist

- Fix indexing, interval bounds, neighbour order, pivot convention and height definition.
- Write the invariant before tracing.
- Count exactly what is asked: comparisons, swaps, probes, queue operations or passes.
- Check empty/singleton inputs, equal keys, disconnected vertices and unreachable targets.
- State expected, amortised and worst-case costs separately.
- For an MSQ, assess each option independently; more than one is not mandatory.

Official question index

These are references to authentic questions, not transcriptions. Open the linked paper and use its exact wording, diagrams and option order. The interactive course checks your answer and reveals an explanation and official key.

GATE DA 2024

Q16 / PDF page 11 / MCQ / ADT matching

Q21 / PDF page 14 / MCQ / Hash insertion

Q32 / PDF page 20 / NAT / Deque trace

Q40 / PDF page 25 / MCQ / Binary-search recurrence

Q44 / PDF page 28 / MCQ / Traversal expansion

Q45 / PDF page 28 / MCQ / Sorting passes

Q51 / PDF page 33 / MSQ / Topological order

Open official paper

Open official answer key

GATE DA 2025

Q18 / PDF page 12 / MCQ / Linear probing

Q27 / PDF page 16 / MSQ / Search prerequisites

Q29 / PDF page 16 / MSQ / Insertion inversions

Q58 / PDF page 31 / MSQ / Shortest-path constraints

Q64 / PDF page 34 / NAT / Stack trace

Q65 / PDF page 34 / NAT / DFS discovery

Open official paper

Open official answer key

GATE DA 2026

Q15 / PDF page 10 / MCQ / Expected quicksort

Q25 / PDF page 17 / MSQ / Tree reconstruction

Q40 / PDF page 25 / MCQ / Reverse reachability

Q49 / PDF page 32 / MSQ / Sorting comparisons

Open official paper

Open official answer key